LOOPS & FUNCTIONS

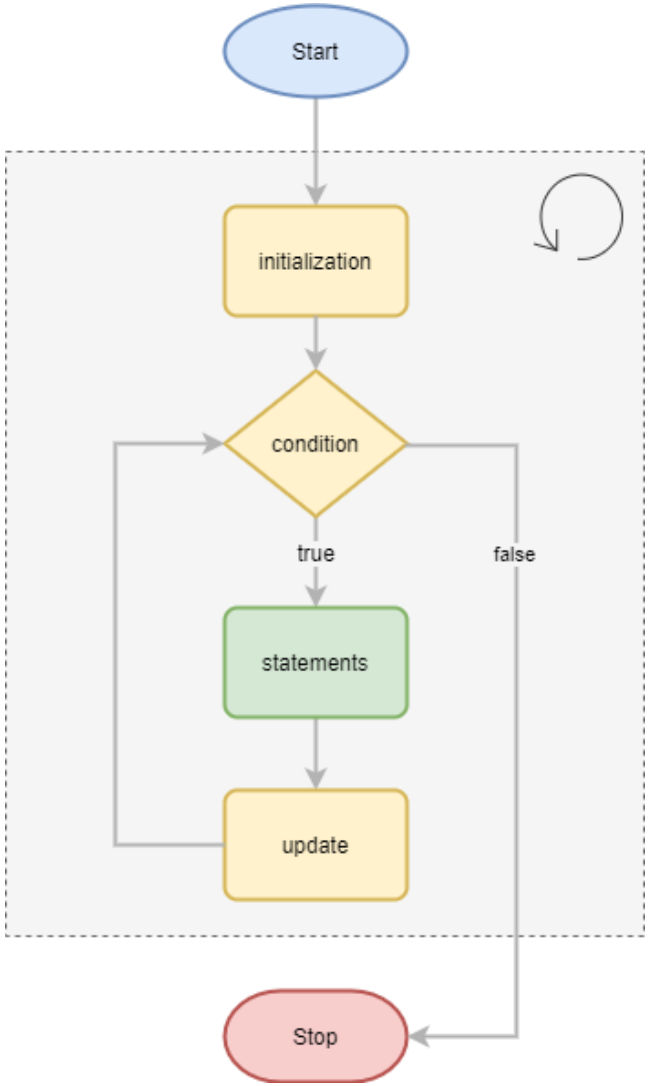
Scripting Programming Languages – Practical lesson 3

Julia Golubitskaya

TOPIC

A **loop** is a block of commands that can be executed repeatedly until a certain condition is met.

A **function** is a standalone block of code that can be called as many times as needed once declared.



TODAY

- Loops in JavaScript
- Functions
  - Arrow functions
  - Default parameters
  - Closure and Lexical environment
- 4 tasks

LOOPS

A **loop** is a block of commands that can be executed repeatedly until a certain condition is met.

Loops are especially useful when you need to run the same code multiple times, each time with a different value.

Main types of loops:

- **for**
- **while**
- **do..while**

THE FOR LOOP

The **for** loop executes a block of commands as long as the given condition is true:

```
for (start; condition; step) {
    // loop body
}
```

One of the more complex but most commonly used loops.

Let's break down each part with an example. The loop below executes *alert(i) for i from 0 to 2* (excluding 3):

```
for (let i = 0; i < 3; i++) {
    alert(i); // outputs 0, then 1, then 2
}
```

THE WHILE LOOP

The **while** loop executes a block of code as long as a specified condition is true.

```
1  while (condition) {
2      // loop body
3  }
```

Example:

```
1  let factorial = 1, i = 2, n = 6;
2  while (i <= n) {
3      factorial *= i;
4      i++;
5  }
6  console.log(factorial);
```

THE DO-WHILE LOOP

The **do-while** loop is often called a post-condition loop because, unlike the previous loops, it executes the block first and then checks the condition. If the condition is true, the block executes again; if false, the loop ends.

Unlike while, do-while executes the code at least once regardless of the condition.

```
do {
    // loop body
} while (condition);
```

FUNCTIONS

A **function** is a standalone block of code (subroutine) that can be called as many times as needed once declared.

- A function may, but does not have to, take parameters.
- Functions in JavaScript are objects, specifically of type Function.

The name determines how we will call the function. Parameters provide values that the function will process. The function consists of a sequence of instructions called the function body. An optional return statement allows returning a value.

```
function name(parameters) {
    //function body
}
```

EXAMPLE: FUNCTIONS

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="utf-8">
5      <title>Example</title>
6  </head>
7  <body>
8
9  <input type='button' value='Click' onclick='writeMessage()' />
10     <script>
11       function writeMessage() {
12          document.write('Push \"F9\" to return to the button');
13         }
14     </script>
15 </body>
16 </html>
```

ARROW FUNCTIONS

There is also a very simple and concise syntax for creating functions called **arrow functions**.

Arrow function expressions are **anonymous** – meaning they don't have a name – unless they are assigned to a variable.

For arrow functions, there is no function keyword. They consist of parameters, followed by the arrow =>, followed by the function body.

A function is anonymous if:

- after the function keyword there is no name (for regular functions).

OR

- it is an arrow function that is not assigned to a variable.

EXAMPLE: ARROW FUNCTIONS

```
1  // create an arrow function with 2 parameters
2  let sum = (x, y) => {
3    return x + y;
4  }
5
6  let number1 = 5, number2 = 36; // function arguments
7
8  // assign the result of sum to the variable answer
9  let answer = sum(number1, number2); // call
10
11 document.write("The sum of x and y is " + answer);
12 // Output: The sum of x and y is 41
13
14
```

ARROW FUNCTIONS & this

Regular function: this is determined by how the function is called (the object before the dot, or the global object).

Arrow function: this does not have its own context. It takes this from the outer (parent) function where it was created.

Consequences:

- Arrow functions cannot be used as constructors (called with new).
- Arrow functions do not have their own arguments object.
- Arrow functions are best used for short callbacks or when you want to preserve the outer this.

DEFAULT PARAMETERS

They allow developers to initialize functions with default values if no arguments are specified when calling the function. Such initialization of function parameters simplifies reading functions, reduces the likelihood of errors, and sets default function behavior. How the function typically works:

```
1  function go(x) { // x - function parameter
2      return x * 10;
3  }
4  go(100); // 100 - function argument
```

If you don't pass an argument to a function that expects to receive it, the function will use the implied value undefined. In this case, go tries to calculate undefined * 10 and gets the result NaN ("not a number").

DEFAULT PARAMETERS

With the addition of default parameters, you can assign any parameter a default value that the function will use instead of undefined if the function is called without an argument. Set a default value for the go function parameter using the equality operator (=). Now when calling the function without an argument, it will assign the value 100 to the variable x and output the calculation result instead of NaN:

```
function go(x = 100) {
  return x * 10;
}
```

You can use any number of default parameters.

DEFAULT PARAMETERS

Note that any primitive or object can be used as a default parameter value.

```
1  function defaultString(string = 'Moscow') {
2      console.log(string);
3  }
4  function defaultBoolean(boolean = true) {
5      console.log(boolean);
6  }
```

In addition you can use the result of a function call as a default parameter:

```
1  function go(x = getRandomNumber(), b = 34) {
2    return x * 10 + b;
3  }
```

CLOSURE & LEXICAL ENVIRONMENT

Closure is when one function is located inside another function. The inner function remembers all the variables that were outside it, even after the outer function has finished running. Think of it as the inner function taking a "snapshot" of its surroundings and carrying that snapshot with it.

Lexical environment is an invisible "list" that is created for each function while it runs. This list simply stores the names of variables and their current values. When a function finishes, this list is usually deleted, but if there is a closure inside it, the list is kept.

Lexical scoping is the rule by which code accesses variables based on where functions are physically written in the source code: wherever a function is written, that is where it looks for its variables (first inside itself, then in the outer function, and so on).

WARM-UP EXAMPLE

```
1  let a = 'global';
2  function outer() {
3      let b = 'outer';
4      function inner() {
5          let c = 'inner';
6          console.log(a); // ?
7          console.log(b); // ?
8          console.log(c); // ?
9      }
10     console.log(a); // ?
11     console.log(b); // ?
12     console.log(c); // ?
13     inner();
14 }
15 outer();
16 console.log(a); // ?
17 console.log(b); // ?
18 console.log(c); // ?
```

CLOSURE & LEXICAL ENVIRONMENT

```
1  function userName() {
2      let name = 'Ivan';
3
4  return function nameOutPut(){
5          console.log(name);
6      };
7  }
8
9  let user = userName();
10 user(); // Ivan
```

Here we call the *userName()* function, which returns the inner function *nameOutPut()*, and save this function in the variable user. When we then call the *user()* function (the corresponding variable actually stores a reference to the *nameOutPut()* function), the name Ivan is output to the console.

At the same time, there is no user variable in the *nameOutPut()* function, so we can conclude that this function can access a variable declared in the outer function *userName()*, even after that function has finished executing.

CLOSURE & LEXICAL ENVIRONMENT

```
1  function counter() {
2      let x = 0;
3      return function() {
4          return x++;
5      }
6  }
7  let plus = counter();
8  console.log(plus()); // 0
9  console.log(plus()); // ?
10 console.log(plus()); // ?
```

As in the previous example, we store a reference to the anonymous inner function returned by the *counter()* function in the variable x.

Note: *This inner function behaves like a closure – we could call it plus, but here it has no name.*

CLOSURE & LEXICAL ENVIRONMENT

```
1  function counter() {
2      let x = 0;
3      return function() {
4          return x++;
5      }
6  }
7  let plus = counter();
8  console.log(plus()); // 0
9  console.log(plus()); // 1
10 console.log(plus()); // 2
```

Note that the value of the variable *x* is not reset to 0 on each call to the returned function (the one stored in variable *x*).

This works because each time the returned function is called, a new execution context is created for it, but there is only one lexical environment for the *counter()* function.

Since the variable *x* is declared in the scope of *counter()*, its value is preserved between calls to the inner function.

WARM-UP EXAMPLE

```
1  let a = 'global';
2  function outer() {
3    let b = 'outer';
4    function inner() {
5      let c = 'inner';
6      console.log(a); // global
7      console.log(b); // outer
8      console.log(c); // inner
9    }
10   console.log(a); // global
11   console.log(b); // outer
12   console.log(c); // error
13   inner();
14 }
15 outer();
16 console.log(a); // global
17 console.log(b); // error
18 console.log(c); // error
```

TASK 1

Checking for a prime number (functions and loops)

- Write a function ***isPrime(n)*** that takes an integer ***n*** and returns ***true*** if the number is prime, and false otherwise. Use a loop to check for divisors. A prime number is a number greater than 1 that is divisible only by 1 and itself.
- Test the function for the numbers 7, 10, and 13, and output the result to the console.

Example of console results:

```
7 is prime: true
10 is prime: false
13 is prime: true
```

TASK 2

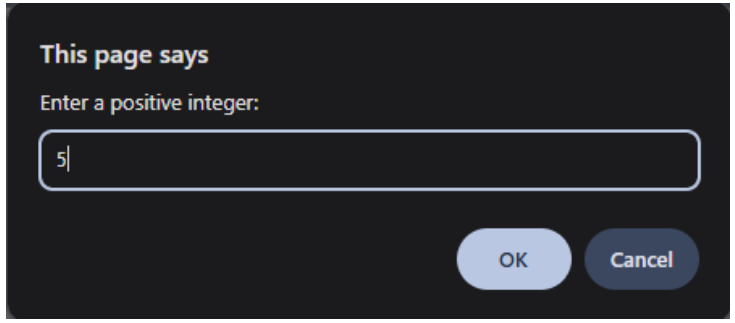
Factorial with input validation (do-while and while loops)

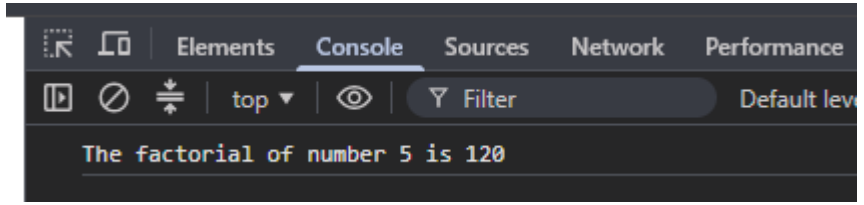
Write a script that:

- uses a *do-while* loop to prompt the user for a positive integer (use prompt);
- checks that the input is a valid number (not NaN) and is greater than zero:
    - if a non-number, negative number, or zero is entered, the program repeats the request;
- after receiving a valid number, the program calculates its factorial using a while loop.

The result is output to the console in the format: "*The factorial of number X is Y*".

Example of results:



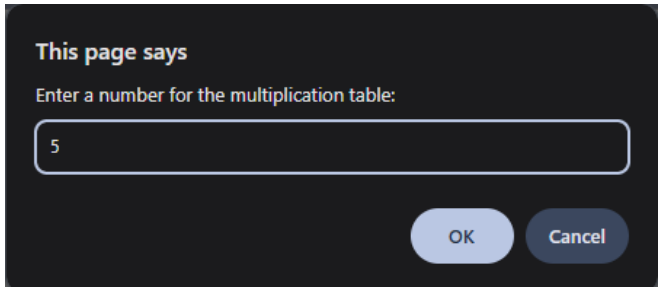


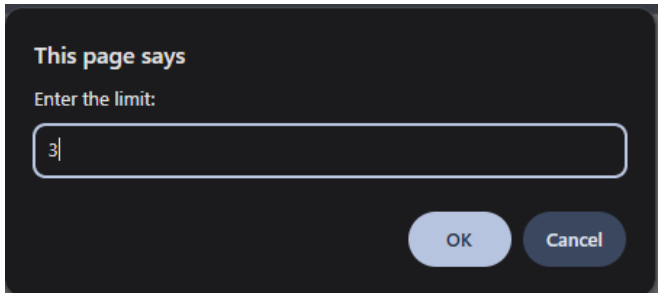
TASK 3

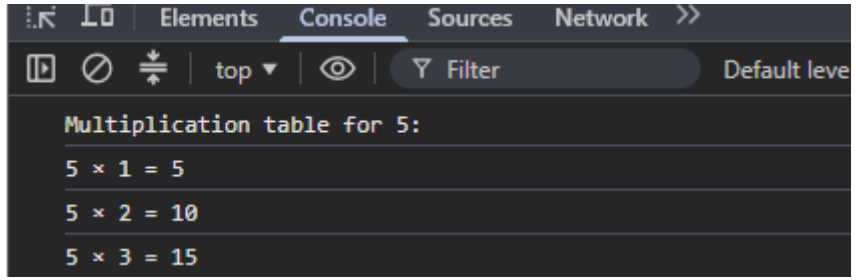
Multiplication table (functions and for loop)

- Write a function ***multiplicationTable(n, limit)*** that outputs to the console the multiplication table for the number n from 1 to limit. Use a **for** loop.
- Then prompt the user for a number and a limit via prompt, convert them to numbers, and call the function.

Example of results:





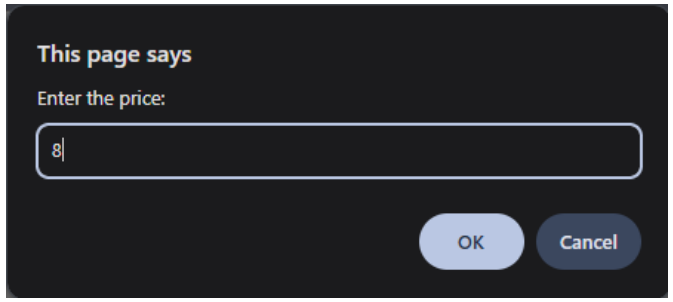


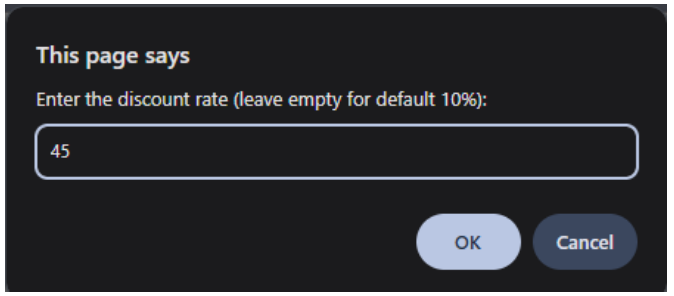
TASK 4

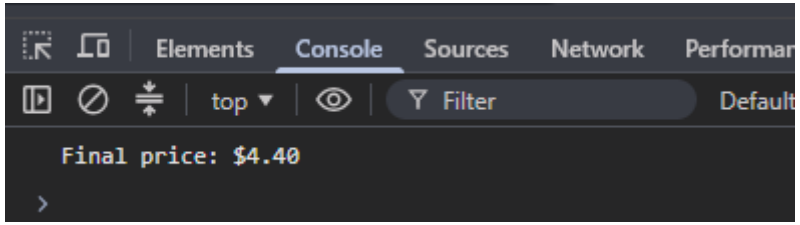
Functions with default parameters

- Write a function ***calculateDiscount*** that calculates and returns the final price after applying a discount. Use the formula: finalPrice = price - (price * discountRate / 100).
- Then ask the user for a price and a discount rate via prompt(), convert them to numbers, call the function, and display the result in the console in the format: "*Final price: $X*".
- If the user does not enter a discount rate, use the default value of 10%.

Example of results:







TEST QUESTIONS

1. What is the difference between for and while loops?
2. Which loop will execute at least once even if the condition is initially false?
3. How does an arrow function differ from a regular function in terms of this behavior?
4. How do you write a function parameter with a default value?
5. What is a closure?
6. What is a lexical environment?
7. Can an arrow function be used as a constructor (called with new)?

THANK YOU FOR ATTENTION

Scripting Programming Languages – Practical lesson 3

Julia Golubitskaya